

Machine Learning and NLP

Alessandro D'Ambrosio

June 2025

Contents

1	Introduzione	1
2	Fondamenti di Machine Learning	2
2.1	Pipeline	2
3	Algoritmi	2
3.1	Naive Bayes	2
3.2	Random Forest	3
3.3	Support Vector Machine	4
3.4	Logistic Regression	4
4	Preprocessing NLP	5
4.1	Bag of Words	5
4.2	Term Frequency – Inverse Document Frequency	6
4.3	Word Embeddings	6
5	Sentiment Analysis	7
6	Emotion Analysis	7
7	Metriche	7

1 Introduzione

Il Machine Learning è un metodo per costruire modelli che apprendono dai dati e fanno previsioni. In sentiment o emotion analysis, usiamo il ML per classificare testi (tweet, recensioni, ecc.) in categorie come positivo/negativo/neutro (sentiment) o gioia, rabbia, tristezza, paura... (emozioni).

2 Fondamenti di Machine Learning

Il Machine Learning (ML) è una branca dell'intelligenza artificiale che si occupa della progettazione e dello sviluppo di algoritmi in grado di apprendere dai dati. Invece di essere esplicitamente programmato, un sistema di ML costruisce un modello matematico a partire da dati di input (training data) per fare previsioni o decisioni senza essere stato programmato per farlo in modo esplicito.

Tipi principali di Machine Learning:

- **Apprendimento supervisionato** – I dati sono etichettati. Esempi: regressione, classificazione.
- **Apprendimento non supervisionato** – I dati non sono etichettati. Esempi: clustering, riduzione dimensionale.
- **Apprendimento per rinforzo** – Un agente impara tramite ricompense o penalità derivanti dalle azioni che compie in un ambiente.

2.1 Pipeline

Le principali fasi per il Machine Learning sono:

- Raccolta Dati Esempio: tweet annotati con emozioni o sentimenti.
- Pre-processing: Pulizia del testo (lowercase, rimozione punteggiatura, stop-word), Tokenizzazione, Stemming/Lemmatizzazione, Vectorizzazione (TF-IDF, Word2Vec, BERT embeddings)
- Train/Test Split Esempio: 80% training, 20% testing.
- Scelta del Modello, Classificatori comuni: Logistic Regression, SVM, Naive Bayes, Transformer-based (BERT)
- Addestramento e Validazione: Cross-validation, Metriche: accuracy, F1-score (utile con classi sbilanciate)
- Test Finale e Deploy Valutazione finale ed esportazione modello (es. joblib) per uso in produzione.

3 Algoritmi

3.1 Naive Bayes

Naive Bayes è un classificatore probabilistico basato sul Teorema di Bayes con l'ipotesi "naive" (semplice) di indipendenza condizionale tra le feature. Il Teorema di Bayes serve a calcolare la probabilità di un evento A dato che un altro evento B è avvenuto, sfruttando le probabilità inverse. La formula generale è:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

lo si può utilizzare anche per la classificazione. Si usa la versione Multinomial che tiene conto della frequenza delle parole (rappresentata tramite il vettore di Feature). Applicando il teorema di Bayes alla categorizzazione delle parole si ottiene:

$$P(C_k|\mathbf{x}) = \frac{P(\mathbf{x}|C_k)P(C_k)}{P(\mathbf{x})}$$

in cui:

- C_k : classe a cui appartiene il testo
- $\mathbf{x}$: vettore delle feature (testo convertito in vettore matematico)
- $P(C_k)$: probabilità a priori della classe
- $P(C_k|\mathbf{x})$: probabilità della classe C_k dato il vettore osservato $\mathbf{x}$ (probabilità a posteriori)
- $P(\mathbf{x}|C_k)$: probabilità di osservare i dati $\mathbf{x}$ se sappiamo che la classe è C_k (probabilità condizionata)

Si assume che le feature siano indipendenti dalla classe ($P(A \cap B) = P(A)P(B) \iff A$ e B sono indipendenti), per cui si calcola:

$$P(\mathbf{x} | C_k) = \prod_{i=1}^V P(w_i | C_k)^{x_i}$$

in cui:

- V è la dimensione del vocabolario (insieme delle parole osservate durante l'addestramento)
- x_i è la frequenza della parola w_i nel documento

$$P(w_i | C_k) = \frac{\sum_j \text{conteggio}(w_j, C_k) + \alpha \cdot V}{\text{conteggio}(w_i, C_k) + \alpha}$$

in cui α è un parametro di smoothing (Laplace smoothing) per evitare probabilità zero

3.2 Random Forest

La Random Forest è un algoritmo di apprendimento supervisionato usato sia per classificazione che per regressione. La Random Forest è un insieme di tanti alberi decisionali, ognuno dei quali lavora su una versione diversa dei dati, e la decisione finale viene presa per voto di maggioranza (classificazione) o media (regressione). Vengono creati diversi alberi decisionali addestrati su un sottoinsieme casuale dei dati (con ripetizione) e un sottoinsieme casuale delle feature ad ogni split per rendere gli alberi meno correlati tra loro. Per classificare un nuovo esempio, ogni albero dà il suo voto (es. “positivo” o “negativo”), e la classe più votata vince. Per la regressione si fa la media di tutti i valori predetti. Ci sono due fonti di casualità che rendono il modello robusto:

- Sampling casuale dei dati: ogni albero vede un campione diverso.
- Scelta casuale delle feature: ogni split interno all'albero considera solo alcune feature, non tutte.

Questo riduce la varianza del modello (cioè la sensibilità a piccoli cambiamenti nei dati) e lo rende meno incline all'overfitting rispetto a un singolo albero decisionale.

3.3 Support Vector Machine

Le Support Vector Machine (SVM) sono algoritmi di classificazione supervisionata che cercano di trovare la miglior frontiera (iperpiano) che separa i dati in classi diverse. Nell'ambito del NLP i punti sono i vettori delle feature $\mathbf{x}$. Si hanno dati espressi in vettori numerici ed etichettati con label +1 oppure -1. Formalmente:

$$(\mathbf{x}_i, y_i) \text{ con } \mathbf{x}_i \in \mathbb{R}^n \text{ e } y_i \in \{-1, +1\}$$

L'iperpiano è definito dall'equazione:

$$\mathbf{x} \cdot \mathbf{w} + b = 0$$

Si pone il margine tra i punti pari a 1, definendo così l'equazione del margine:

$$y_i(\mathbf{w} \cdot \mathbf{x} + b) \geq 1 \quad \forall i$$

Applicando le formule di geometria analitica della distanza tra un punto e una retta, la distanza tra i due SVM più vicini sarà $\frac{2}{\|\mathbf{w}\|}$. Da cui si ottiene un problema di ottimizzazione

$$\min_{w,b} \frac{1}{2} \|\mathbf{w}\|^2$$

con vincoli

$$y_i(\mathbf{w} \cdot \mathbf{x} + b) \geq 1 \quad \forall i$$

3.4 Logistic Regression

È un modello di classificazione, nonostante il nome. Serve a predire la probabilità che un input appartenga a una delle due classi (es. sentiment positivo o negativo). Stima la probabilità che un'osservazione appartenga alla classe positiva (es. sentiment = positivo), restituendo un output tra 0 e 1, usando la funzione sigmoide. Sia $\mathbf{x}$ il vettore delle feature (es. TF-IDF di un testo), $\mathbf{w}$ i pesi. Si calcola:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad \text{con } z = \mathbf{w}^\top \mathbf{x} + b$$

in cui $\hat{y}$ è la probabilità che l'input appartenga alla classe 1 e $\sigma(\mathbf{z})$ è la funzione sigmoide

Per l'addestramento si devono ottimizzare i pesi $\mathbf{w}$ e il bias $\mathbf{b}$, si utilizza una funzione di costo (loss) Binary Cross-Entropy da minimizzare.

$$\mathcal{L}(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

I pesi vengono aggiornati con:

$$w_i \leftarrow w_i - \eta \frac{\partial \mathcal{L}}{\partial w_i}$$

analogamente anche il bias:

$$b \leftarrow b - \eta \frac{\partial \mathcal{L}}{\partial b}$$

Derivando la funzione di loss per il bias e i pesi, si ottiene:

$$w_i \leftarrow -\eta(\hat{y} - y)x_i$$

$$b \leftarrow b - \eta(\hat{y} - y)$$

In questo modo se la probabilità viene sovrastimata i pesi verranno abbassati, mentre se viene sottostimata vengono aumentati.

4 Preprocessing NLP

Il preprocessing prepara il testo grezzo per essere convertito in un formato numerico utilizzabile dai modelli ML. Ci sono diverse fasi prima della conversione dei documenti in vettori di feature:

1. **Tokenizzazione** Si scompone il testo in unità, ovvero parole oppure sottoparole
2. **Lowercasing** Si convertono tutte le lettere in minuscolo
3. **Rimozione simboli superflui** come punteggiatura o simboli
4. **Rimozione Stopword**, ovvero parole comuni che non forniscono informazioni come articoli o preposizioni
5. **Stemming** si riducono le parole a una radice comune
6. **Lemmatizzazione** Porta una parola alla sua forma canonica

4.1 Bag of Words

Bag of Words(BoW) è il metodo più semplice e fondamentale per convertire un testo in un vettore numerico, senza considerare la struttura grammaticale né il significato. Si costruisce un vocabolario estraendo le parole uniche, ovvero i token. Ogni documento viene rappresentato come vettore di dimensione $|V|$ ogni componente del vettore rappresenta quante volte la parola w_i compare nel documento.

4.2 Term Frequency – Inverse Document Frequency

Term Frequency – Inverse Document Frequency (TF-IDF) è una pesatura intelligente delle parole. Migliora Bag-of-Words penalizzando le parole troppo comuni e valorizzando le parole caratteristiche di un documento. Per ogni parola w_i si calcola il valore $TF - IDF$

$$TF - IDF(w, d) = TF(w, d) \cdot IDF(w, d)$$

in cui TF e IDF sono due funzioni matematiche:

$$TF(w, d) = \frac{\text{conteggio di } w \text{ in } d}{\text{numero totale di parole in } d}$$

calcola quante volte la parola w compare in d

$$IDF(w, d) = \log\left(\frac{N}{1 + DF(w)}\right)$$

in cui N è il numero totale di documenti e $DF(w)$ è il numero di documenti che contiene la parola w . In questo modo vengono penalizzate le parole molto comuni nei documenti, l'idea è che una parola molto comune non sia determinante nella categorizzazione.

4.3 Word Embeddings

Un embedding è una rappresentazione vettoriale densa di una parola. L'idea è che parole semanticamente simili hanno vettori vicini, dunque si può usare la geometria per catturare il significato. Gli embedding si ottengono tramite un modello (Word2Vec, GloVe, FastText) che impara dai co-occurrence patterns delle parole nei testi. Due parole co-occorrono quando appaiono insieme nello stesso contesto (es. nella stessa frase, finestra di parole, documento, ecc.) Ogni modello differisce per come cattura la semantica. Word2Vec è un modello predittivo: non si basa su conteggi espliciti, ma impara a predire le parole nel contesto usando una piccola rete neurale.

- Il modello Skip-Gram, dato un termine (es. “film”) il sistema cerca di predire le parole vicine (es. “bello”, “commovente”).
- Il CBOW, fa l'opposto: dato il contesto, predice la parola centrale.

Questo lo rende molto efficiente e bravo a catturare relazioni semantiche e sintattiche locali, come analogie tipo “re - uomo + donna = regina”.

GloVe, invece, è un modello statistico: costruisce una matrice di co-occorrenza globale tra tutte le parole del corpus, poi la fattorizza per ottenere i vettori. Tiene conto di quante volte due parole appaiono insieme in tutto il testo. È particolarmente utile per catturare relazioni semantiche su larga scala, anche tra parole che appaiono raramente ma in contesti simili.

Per concludere, Word2Vec impara da previsioni locali (predire il contesto), mentre GloVe apprende da statistiche globali (conteggi di co-occorrenze).

5 Sentiment Analysis

6 Emotion Analysis

7 Metriche